Fiche de révision — Stream Processing avec Apache Kafka

ESGI · 5e année · Data Engineering · Séances 1 à 8

Synthèse des notions à maîtriser pour l'examen (QCM, Vrai/Faux, réponses multiples). Révise chaque séance point par point.

Séance 1 — Des pipelines au log distribué

À maîtriser

- Un pipeline fait circuler la donnée en continu, en quasi temps réel.
- Sans point central : N×M liaisons fragiles ; avec un log central : N+M (chaque système se branche une fois).
- Deux modèles historiques : file point-à-point (1 message → 1 seul consommateur, supprimé après lecture) vs publication/abonnement (1 message → tous les abonnés).
- Limites des brokers classiques (ActiveMQ/JMS, RabbitMQ, IBM MQ, TIBCO) : rétention courte, pas de rejeu, montée en charge difficile, couplage.
- Kafka, c'est un log : append-only, ordonné (offset), immuable (on relit), persisté sur disque.
- Un topic = un log ; l'offset identifie la position de chaque événement.
- Ce que le log apporte : découplage, rétention, rejeu, multi-consommateurs, débit élevé, ordre garanti (par partition).

Mots-clés : topic, offset, log append-only, producteur/consommateur, rejeu. Environnement : Docker Compose + Kafka UI (8080), broker KRaft (9092), Java 17+.

Séance 2 — Anatomie distribuée : partitions, brokers, KRaft

À maîtriser

- Un topic est découpé en partitions ; chaque partition est un log ordonné et indépendant.
- Le triplet (topic, partition, offset) adresse un message ; les offsets ne sont pas comparables entre partitions.
- L'ordre n'est garanti qu'au sein d'une partition, jamais à l'échelle du topic.
- Routage : avec une clé → hash(clé) % n (même clé → même partition) ; sans clé → réparti.
- La partition est l'unité de débit, de scalabilité et de parallélisme.
- Un cluster = plusieurs brokers ; chaque partition a 1 leader + des réplicas (facteur de réplication).
- KRaft : métadonnées sans ZooKeeper, via un quorum de contrôleurs (Raft) ; un nœud peut être broker, controller, ou les deux.

Mots-clés & commandes : --partitions, --replication-factor, kafka-topics.sh --describe (Leader / Replicas / Isr), KAFKA_PROCESS_ROLES, KAFKA_CONTROLLER_QUORUM_VOTERS.

Séance 3 — Réplication & durabilité

À maîtriser

- Le producteur n'écrit qu'au leader ; les réplicas recopient son log en continu (fetch).
- L'ISR = les réplicas synchronisés et à jour ; seuls eux peuvent devenir leader.
- À la panne du leader, un réplica de l'ISR est promu automatiquement, sans perte des écritures confirmées.
- acks : 0 (aucun accusé, perte possible), 1 (leader seul), all (tout l'ISR, le plus sûr).
- Garantie de non-perte : acks=all + min.insync.replicas (ISR sous le seuil → écriture refusée, NOT_ENOUGH_REPLICAS).
- unclean.leader.election.enable (défaut false) arbitre disponibilité vs durabilité ; ELR (Kafka 4.x) sécurise encore l'élection.
- Rétention : cleanup.policy=delete (purge par temps/taille) vs compact (dernière valeur par clé).

Mots-clés : ISR, acks, min.insync.replicas, retention.ms / retention.bytes, cleanup.policy (delete / compact), unclean.leader.election.enable.

Séance 4 — Java & API Producer / Consumer

À maîtriser

- Client officiel natif JVM ; Kafka Streams, Interactive Queries et TopologyTestDriver n'existent qu'en Java (17/21).
- Projet Maven, une seule dépendance : kafka-clients.
- Idiomes attendus : record, var, lambda, try-with-resources.
- Producteur : Properties → KafkaProducer → ProducerRecord(topic, clé, valeur) → send().
- send() est asynchrone (retourne un Future) ; un callback donne partition/offset ; flush() / close() vident le tampon.
- Consommateur : Properties + group.id → subscribe → boucle poll(Duration).
- auto.offset.reset : earliest (tout) / latest (nouveaux) ; l'offset commité est la mémoire du groupe.

Config clés : bootstrap.servers, key/value.serializer, acks, group.id, auto.offset.reset, poll(Duration).

Séance 5 — Consumers avancés : groupes, rebalance, offsets

À maîtriser

- Routage par murmur2(clé) % n ; l'ordre par clé ne tient que dans une partition.
- Bien choisir la clé : attention au skew (faible cardinalité → partitions chaudes) ; changer n casse le mapping clé→partition.
- Dans un groupe : 1 partition → au plus 1 consommateur ; le débit d'un groupe plafonne au nombre de partitions.
- Rebalance : déclenché quand un consommateur rejoint/part/tombe ou que des partitions sont ajoutées ; stratégies range, round-robin, sticky, cooperative-sticky ; static membership (group.instance.id).
- KIP-848 : nouveau protocole piloté côté serveur, incrémental et coopératif (opt-in, group.protocol=consumer).
- Offset commité = prochain message à lire, stocké dans __consumer_offsets ; LAG = LOG-END-OFFSET − CURRENT-OFFSET.
- Commit après traitement = at-least-once ; commit avant = at-most-once.

Mots-clés & commandes : kafka-consumer-groups.sh --describe, enable.auto.commit, commitSync(), LAG, __consumer_offsets.

Séance 6 — Sémantiques de livraison & exactly-once

À maîtriser

- Trois sémantiques : at-most-once (perte), at-least-once (doublons), exactly-once (aucun, à l'intérieur de Kafka).
- Idempotence du producteur : PID + numéro de séquence par partition ; activée par défaut depuis Kafka 3.0 (acks=all, max.in.flight ≤ 5).
- Transactions : transactional.id, écriture atomique sur plusieurs partitions/topics ; transaction coordinator via __transaction_state ; zombies exclus par epoch fencing.
- Cycle : initTransactions() → beginTransaction() → send() → sendOffsetsToTransaction() → commit/abort.
- consume-transform-produce = exactly-once à l'intérieur de Kafka (pas vers un système externe).
- isolation.level=read_committed : ne lit que ce qui est validé (jusqu'au LSO).
- Ouverture : share groups (KIP-932) permettent consos > partitions, avec acquittement par message.

Mots-clés : enable.idempotence, transactional.id, read_committed, PID / epoch, EOS, LSO.

Séance 7 — Sérialisation & Schema Registry

À maîtriser

- Un message n'est qu'une suite d'octets : le schéma est le contrat entre producteur et consommateur.
- Formats : JSON (lisible, volumineux, évolution fragile), Avro (binaire compact, .avsc, évolution excellente — référence de Kafka), Protobuf (.proto).
- Champ optionnel = union ["null", "type"] avec une valeur par défaut.
- Schema Registry (composant Confluent, API REST, topic _schemas) : stocke et versionne les schémas, attribue un ID, applique la compatibilité.
- Trame : magic byte + ID de schéma (4 octets) + payload ; côté producteur on remplace juste le sérialiseur. Sujets <topic>-value / <topic>-key.
- Compatibilité : BACKWARD (défaut : ajouter avec défaut / supprimer), FORWARD, FULL, NONE ; un changement incompatible est refusé (409 Conflict).

Mots-clés & commandes : schéma .avsc, KafkaAvroSerializer, schema.registry.url (8081), /subjects, BACKWARD / FORWARD / FULL.

Séance 8 — Kafka Connect

À maîtriser

- Déplacer des données entre Kafka et l'extérieur sans écrire de code : on déclare l'intégration (JSON) au lieu de la coder.
- Source : lit un système externe et publie dans Kafka. Sink : consomme des topics et écrit vers un système externe.
- Architecture : workers (processus JVM), connecteurs, tasks (parallélisme) ; en mode distribué, l'état vit dans des topics internes (cluster sans état local).
- Un connecteur = une config JSON (connector.class, tasks.max, topic…) envoyée à l'API REST (POST /connectors, port 8083).
- Convertisseurs : StringConverter / JsonConverter (sans registre) ; AvroConverter réutilise le Schema Registry (S7).
- SMT (Single Message Transforms) : transforment les messages un par un (RegexRouter, InsertField, MaskField, ValueToKey).
- Écosystème : JDBC, Debezium (CDC), S3, Elasticsearch ; MirrorMaker 2 ; DLQ, tasks.max. La logique riche sur les flux, c'est Kafka Streams.

Mots-clés & commandes : POST /connectors (8083), connector.class, tasks.max, value.converter, SMT, source / sink.